

How Computers Work: CPU, Memory, Storage

Part I · Foundations — understand one machine before you connect many.

LEARNING OBJECTIVES

- Describe the three core jobs of a computer — compute, remember, store — and how they connect.
- Explain what a CPU does, what a core and a thread are, and why the CPU spends much of its life *waiting*.
- Draw the **memory hierarchy** and explain the speed-size-cost trade-off between each layer.
- Explain why RAM is fast but volatile, and why disks are persistent but slow.
- Explain the huge difference between **sequential** and **random** disk access, and why databases care.
- Connect these hardware facts to real system-design decisions: caching, in-memory stores, log-structured storage, and CDNs.
- Explain what limits a single machine, motivating the move to many machines in later parts.

REAL-WORLD ANALOGY — A CHEF AT A WORKBENCH

Picture a chef (the **CPU**) working at a bench. The few ingredients right in their hands are like **registers** — instant to use but tiny. The cutting board within arm's reach is **cache** — very fast, still small. The countertop is **RAM** — roomy and quick, but cleared at the end of the day (volatile). The pantry across the kitchen is the **SSD/hard disk** — huge and permanent, but you must walk to it. And a delivery from another city is the **network** — enormous capacity, but you wait a long time. A great chef keeps whatever they're using *close*. That single instinct — keep hot data near the top — is the seed of caching, and of much of this book.

2.1 Why a system designer starts with one machine

This book is about connecting many machines, so why begin inside one? Because **every large-scale technique is a reaction to how a single computer behaves**. We cache because moving data down the hierarchy is thousands of times slower. We prefer sequential writes because disks hate random jumps. We put servers near users because distance is measured in milliseconds. If you understand the raw speeds of hardware, the rest of system design stops feeling like arbitrary rules and starts feeling like obvious consequences.

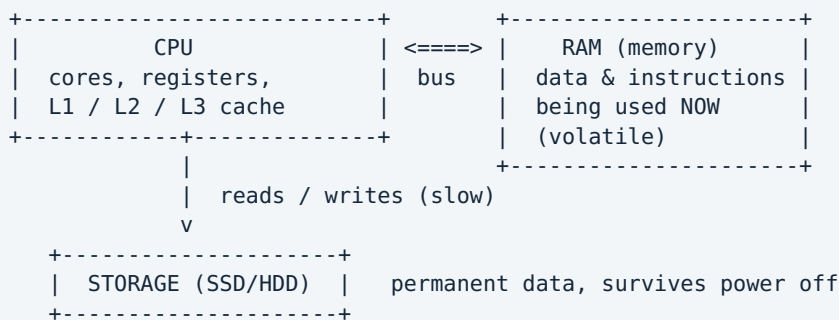
2.2 The three jobs: compute, remember, store

At its heart, every computer does three things, handled by three kinds of hardware:

- **Compute** — the **CPU** (Central Processing Unit) does the actual work: arithmetic, comparisons, moving data around.
- **Remember (short-term)** — **memory (RAM)** holds the data and instructions the CPU is working on *right now*. Fast, but forgotten when power is lost.
- **Store (long-term)** — **storage (SSD/HDD)** keeps data permanently, even when the machine is off. Big, but slow.

They are wired together by a **bus** (the internal highway that carries data between them). This basic layout — a CPU that fetches instructions and data from memory, with persistent storage behind it — is called the **von Neumann architecture**, and nearly every computer you use follows it.

THE BASIC MACHINE (von Neumann model)



WHAT PROBLEM DOES UNDERSTANDING THIS SOLVE?

Beginners often imagine data is “just there” and every operation is equally fast. In reality, the CPU is almost always faster than the data can reach it. Knowing where data lives — and how far it must travel — is what lets you predict whether a system will be fast or slow *before* you build it.

2.3 The CPU: fast, but usually waiting

The CPU executes instructions — billions per second. A few terms:

- **Clock speed** (e.g. 3 GHz) — roughly how many basic steps per second the CPU can take (3 GHz $\approx$ 3 billion cycles/second). Higher is faster, but it stopped rising much a while ago because of heat and power limits.
- **Core** — an independent processing unit. A 8-core CPU can truly do 8 things at the exact same instant. Since clock speeds stopped climbing, chips got **more cores** instead of faster ones — which is why software now must do many things in parallel to go faster.
- **Thread** — a single sequence of instructions. One core runs one thread at a time (some cores fake two via “hyper-threading”). Threads are how one program does several things at once.

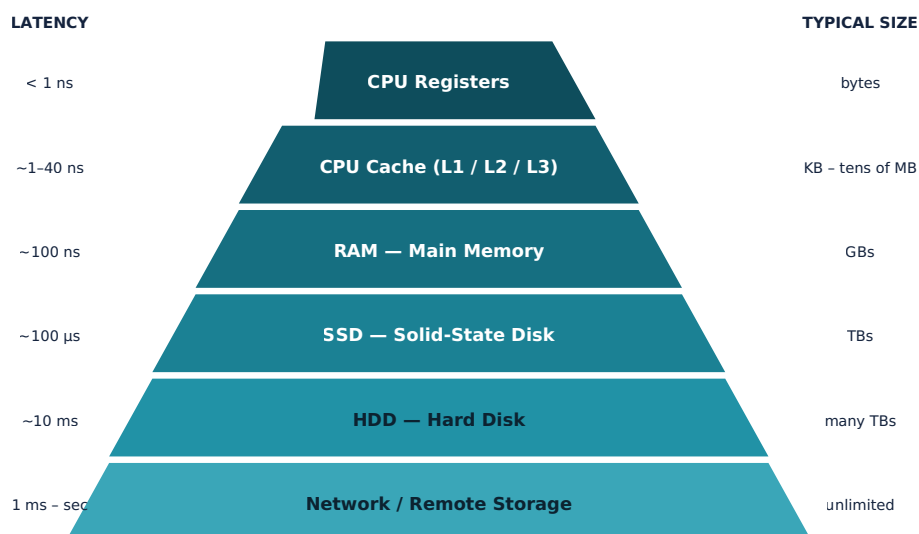
KEY INSIGHT — THE CPU IS STARVED FOR DATA

A modern CPU can execute an instruction in well under a nanosecond, but fetching a piece of data from main memory takes around **100 nanoseconds**. In the time the CPU waits for *one* memory read, it could have done **hundreds** of calculations. So the CPU spends much of its life *stalled*, waiting for data to arrive. This single fact — compute is cheap, **fetching data is expensive** — is why the memory hierarchy exists, and why caching is the most repeated idea in all of system design.

To hide this waiting, CPUs keep tiny, ultra-fast **caches** (L1, L2, L3) right on the chip. When the CPU needs data, it checks L1 first (fastest, smallest), then L2, then L3, then finally main memory. A **cache hit** (data was already close) is fast; a **cache miss** (must go further down) is slow. You will meet this exact pattern again at the application level with Redis and CDNs — it is the same idea, scaled up.

2.4 The memory hierarchy — the most important diagram in the book

Storage technologies form a **hierarchy**: as you go down, each layer is bigger and cheaper per byte, but dramatically slower. As you go up, each layer is smaller and pricier, but blindingly fast. There is no single “best” layer — real systems use all of them, keeping the data they touch most often as high up as possible.



Top = tiny, blazing fast, expensive. Bottom = huge, slow, cheap. Good design keeps the data you touch most often near the top.

KEY INSIGHT — EACH STEP DOWN IS A BIG JUMP, NOT A SMALL ONE

The gaps are not 2× — they are often 100× to 1,000,000×. RAM is roughly **1,000×** slower than cache; an SSD is roughly **1,000×** slower than RAM; an old hard-disk seek is another **100×** slower than that. Because the cliffs are so steep, moving a piece of data *one layer up* can make an operation hundreds of times faster. That is the entire economic case for every cache you will ever add.

Making the numbers human

Nanoseconds are hard to feel. So let's stretch time: imagine one CPU-cache access took **1 second**. Then everything else scales up like this:

Real operation	Real time (approx.)	If cache = 1 second...
CPU cache access	~1 nanosecond	1 second
Main memory (RAM) read	~100 nanoseconds	~1.5 minutes
SSD random read	~100 microseconds	~1 day
Network round trip (same data center)	~0.5 milliseconds	~6 days
Hard-disk seek	~10 milliseconds	~4 months
Network round trip (across the world)	~150 milliseconds	~5 years

Read that last row again. If touching cache feels like one second, then a request crossing the planet feels like **five years**. That is why we never casually reach across continents in a hot code path, and why CDNs and regional replicas (later chapters) exist: to turn a “five-year” trip back into a “six-day” one.

2.5 RAM: fast, roomy, but forgetful

RAM (Random-Access Memory) is the computer's short-term working memory. Any byte can be read in about the same ~100 ns regardless of location (that's the “random-access” part — no penalty for jumping around, unlike a hard disk). It is measured in gigabytes and is fast enough that most software treats it as “where the data lives while I work.”

Its one catch is that RAM is **volatile**: cut the power and everything in it vanishes. That is why databases must eventually write to disk, and why a crash can lose anything that was only in memory.

WHERE THIS SHOWS UP LATER

Systems like **Redis** and **Memcached** keep data *in RAM* on purpose, trading durability risk for microsecond speed. In-memory databases can be 1,000× faster than disk-based ones precisely because they never pay the storage-layer cost — the same hierarchy trade-off, now a product decision.

2.6 Storage: permanent, but slow — and it hates random access

Storage keeps data forever. There are two main kinds, and the difference matters enormously for databases.

Hard Disk Drive (HDD)	Solid-State Drive (SSD)
Spinning magnetic platters with a moving read/write head.	Flash memory chips — no moving parts.

Cheap per byte; great for huge cold data.

Far faster; now standard for anything performance-sensitive.

Random access is very slow — the head must physically move (“seek”) and wait for the platter to spin (~10 ms per jump).

Much better at random access (no moving parts), but still organized in blocks, so access patterns still matter.

Sequential vs random access — the pattern that shapes databases

How you read data can matter more than how much you read. Reading blocks **in order** (sequential) lets the disk stream data smoothly at high speed. Reading blocks **scattered all over** (random) forces constant jumping, and on an HDD each jump costs a slow physical seek.

SEQUENTIAL read (FAST): blocks read in order, one smooth pass
[b1][b2][b3][b4][b5][b6] ... ==> high MB/s, disk stays “in the zone”

RANDOM read (SLOW on HDD): jump all over the platter
[b5].....[b1]...[b9]....[b3] ==> each jump = seek + rotational wait

KEY INSIGHT — THIS IS WHY DATABASES ARE BUILT THE WAY THEY ARE

Because sequential disk access is dramatically faster than random access, storage systems are engineered to **turn random work into sequential work**. It is the reason for:

- **Append-only logs** — writing new data to the end of a file (pure sequential). This is exactly how **Kafka** achieves huge throughput on ordinary disks.
- **LSM-trees** (used by Cassandra, RocksDB) — buffer writes in memory, then flush them to disk in big sequential batches.
- **B-tree indexes** (used by most relational databases) — carefully structured so that lookups touch as few disk blocks as possible.

When you study databases in Part III, remember: their cleverest tricks all exist to dodge the cost of random disk access you just learned about here.

2.7 How hardware truths drive system design

Everything above turns directly into design rules you will use for the rest of the book:

Hardware fact	The design rule it creates
Fetching data down the hierarchy is 100–1,000,000× slower.	Cache hot data as high up as possible (CPU cache → RAM → Redis → CDN).
RAM is ~1,000× faster than disk.	Keep the “working set” of hot data in memory ; use in-memory stores for speed-critical paths.
Random disk access is far slower than sequential.	Prefer append-only / sequential writes; this shapes logs, queues, and databases.
Crossing the world costs ~150 ms per round trip.	Put data and servers near users (CDNs, regional replicas); minimize round trips.
RAM is volatile; power loss erases it.	

Persist important data to disk; never treat an in-memory-only write as “saved.”

2.8 The limits of one machine (and why we’ll need many)

A single computer can do more than one thing at a time using **processes** and **threads**:

- A **process** is a running program with its own private memory. Two processes are well isolated but talking between them is more expensive.
- A **thread** is a sequence of execution *inside* a process; threads in one process share memory, so they’re cheaper to coordinate but can interfere with each other (bugs called race conditions).

More cores and threads let a machine do more in parallel — but there are hard ceilings. A single server has a finite number of cores, a fixed amount of RAM, limited disk and network bandwidth, and a limited number of open network connections it can juggle.

COMMON MISUNDERSTANDING — “JUST ADD MORE THREADS”

Adding threads does not always make things faster. If the real bottleneck is a slow disk or network (an **I/O-bound** task), extra threads just sit waiting too. If the task is **CPU-bound**, you can’t usefully run more busy threads than you have cores. And every thread costs memory and adds **context-switching** overhead (the CPU’s cost to pause one thread and resume another). Beyond a point, more threads make a system *slower*.

KEY INSIGHT — THE CEILING IS WHY THIS BOOK EXISTS

Historically, getting a single server to handle even **ten thousand simultaneous connections** (the famous “C10K problem”) was hard. No matter how big you build one machine, you hit a wall on cores, memory, bandwidth, or reliability — and it’s still a single point of failure. That wall is exactly why the next parts of the book scale **out** to many machines, and why nearly every hard problem ahead — load balancing, replication, consistency — comes from coordinating many computers that each behave like the one you just studied.

Common mistakes

WATCH OUT

- Treating memory access as “free” and imagining RAM and disk are similarly fast — they differ by $\sim 1,000\times$.
- Ignoring **random vs sequential** access when designing storage or choosing a database.
- Assuming more threads always means more speed, even for I/O-bound or single-core-bound work.
- Believing a bigger single machine scales forever (the vertical-scaling ceiling from Chapter 1).
- Forgetting RAM is volatile — counting an in-memory-only write as durably saved.
- Reaching across regions or continents inside a hot path without realizing each trip can cost ~ 150 ms.

Interview questions

1. Explain the memory hierarchy. Why does caching work, and where does it appear at both the hardware and application level?
2. Why is random access on a hard disk so much slower than sequential access? What is the physical cause?
3. Roughly how much slower is an SSD read than a RAM read, and why does that gap influence database design?
4. What is the difference between a process and a thread? When would adding threads *not* speed up a program?
5. Why do systems like Kafka and Cassandra favor append-only or sequential writes?
6. Why is RAM volatility important to a database's design?
7. What physical limits stop a single server from scaling forever?
8. A cross-continent round trip is ~150 ms. What design techniques reduce the impact of that distance?

Hands-on exercises

1. Find your own machine's specs: number of cores, amount of RAM, cache sizes, and whether the disk is SSD or HDD. Write them down.
2. Using the latency table, calculate roughly how many RAM reads (~100 ns) could complete in the time of a single HDD seek (~10 ms).
3. Convert the SSD read and cross-country network latencies to the "cache = 1 second" human scale, and check them against the table.
4. You have 16 GB of RAM. Decide whether a 2 GB dataset can be held fully in memory, and what you'd do instead for a 500 GB dataset.
5. In two sentences, explain why an append-only log can be fast even on a cheap hard disk.

Mini project

MEASURE THE HIERARCHY YOURSELF

In any language you like, write a small program that does three timed tasks and prints how long each takes: **(a)** sum every element of a large array already in memory; **(b)** read a large file (say 1 GB) from start to finish *sequentially*; **(c)** read the same file at 10,000 *random* offsets. Compare the throughput (MB/s) of the sequential vs random reads, and compare both against the in-memory sum. You should see, with your own eyes, that **memory » sequential disk » random disk**. Keep the numbers — they make the rest of this book concrete.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE HARDWARE LAYER

Open your design notebook from Chapter 1. Estimate the **"hot" working set**: roughly how much data do actively-online users touch at once (their profiles, recent feed items)?

Decide, using this chapter's numbers, what should live in **RAM** (fast cache) versus on **disk**. Then label two access patterns in your design: one that is naturally **sequential** (e.g. appending new posts to a log) and one that is naturally **random** (e.g. fetching one specific user by ID). You will build directly on these labels when we reach caching (Chapter 8) and databases (Part III).

Chapter summary

- A computer does three jobs: **compute** (CPU), **remember short-term** (RAM), and **store long-term** (SSD/HDD), wired together by a bus — the von Neumann model.
- The CPU is extremely fast but is usually **waiting on data**; on-chip caches (L1/L2/L3) exist to hide that wait. Compute is cheap; fetching data is expensive.
- The **memory hierarchy** trades speed for size and cost. Each step down is 100×–1,000,000× slower, which is the entire justification for caching.
- **RAM** is fast and roomy but **volatile**; **disks** are permanent but slow, and **hard disks punish random access** severely.
- **Sequential access** is far faster than random access — the reason logs, queues, and databases are engineered to write sequentially.
- These hardware truths become design rules: cache hot data, keep the working set in memory, prefer sequential writes, and put data near users.
- A single machine has hard ceilings (cores, RAM, bandwidth, connections) and is a single point of failure — which is why the book now moves toward **many** machines.

COMING NEXT

Chapter 3 — Networking Fundamentals: TCP, UDP, IP. Once we have many machines, they must talk to each other. We'll learn how data actually travels between computers, why some connections are reliable and others fast, and how the network's own delays (which you just met at the bottom of the hierarchy) shape every distributed design.